



TRABAJO ESCRITO

GymSuppStore



Juan Pablo Valverde Barreiro

**Facultad de Ingeniería, Universidad Corporación Universitaria
del Huila**

Programa Ingeniería de Sistemas

Docente: José Miguel Llanos

17 de mayo de 2026

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8360699

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA

"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado, aplicando todos los requisitos de las normas ISO implementadas en sus sedes Neiva y Pitalito"



Contenido

INTRODUCCION	3
OBJETIVOS	4
2.1 Objetivo General	4
2.2 Objetivos Específicos	4
DESARROLLO	5
3.1 Marco Teórico.....	5
3.1.1 Calidad de software ISO/IEC 25010	5
3.1.2 Procesos de Prueba ISO/IEC 29119	5
3.1.3 QA VS QC	5
3.1.4 Métricas de Calidad	7
3.2 Descripción de la Aplicación GymSupp Store.....	7
3.3.1 Estrategia y Diseño	8
3.3.2 Resultados de Cobertura	8
3.4 Pruebas Funcionales con Selenium WebDriver.....	9
3.4.1 POM (Patron Page Object Model)	9
3.4.2 Casos de Prueba Ejecutados.....	10
3.5 Pruebas de Rendimiento con JMeter	10
3.5.1 Plan de Prueba	10
3.6 Análisis de Calidad con SonarQube	11
3.6.1 Configuración	11
3.6.2 Resultados del Dashboard	12
3.6.3 Hallazgos y Plan de mejora	12
3.7 Docker y CI/CD con GitHub Actions.....	13
3.7.1 Docker Compose	13
3.7.2 Pipeline GitHub Actions.....	13
Resultados Obtenidos	15
CONCLUSION	16

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8360699

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA

"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado, aplicando todos los requisitos de las normas ISO implementadas en sus sedes Neiva y Pitalito"



INTRODUCCION

Para empezar, la calidad de software es una serie de pruebas en testeos que cumplen requisitos funcionales, rendimiento, seguridad y experiencia del usuario. En un entorno donde las aplicaciones web deben responder a millones de usuarios con alta disponibilidad y seguridad, garantizando la calidad desde las etapas tempranas del desarrollo no es opcional sino una necesidad estratégica.

En este presente trabajo se documenta el desarrollo e implementación de un sistema integral de gestión de pruebas para la aplicación web GymSupp Store, un sistema de gestión de inventario para tiendas de suplementos deportivos desarrollado en Java con Spring Boot. El proyecto integra herramientas modernas de QA/QC incluyendo JUnit 5, Selenium WebDriver, JMeter y SonarQube mediante Docker y automatizado en GitHub Actions.

La aplicación GymSupp Store implementa funcionalidades de autenticación de usuarios, gestión CRUD de productos, validación de datos y una interfaz web desarrollado con Thymeleaf. Sobre esta base se construyó una suite completa de pruebas automáticas que cubre desde las unidades más pequeñas del código hasta el comportamiento de la interfaz bajo condiciones de carga.

El documento se va a desarrollar el marco teórico, descripción de la aplicación, implementación de cada tipo de prueba, resultados obtenidos, conclusiones y referencias bibliográficas.



OBJETIVOS

2.1 Objetivo General

Aplicar de forma integral los conceptos de calidad de software, estándares internacionales (ISO/IEC 25010, ISO 9001, CMMI, ISO/IEC 29119) y herramientas modernas de QA/QC en el desarrollo y validación de la aplicación web GymSupp Store.

2.2 Objetivos Específicos

1. Implementar pruebas unitarias con Junit 5 y Mockito alcanzando una cobertura mínima del 80%
2. Desarrollar pruebas funcionales de interfaz de usuario con selenium webdriver utilizando el patrón page object model (POM) en Chrome y Firefox
3. Ejecutar pruebas de rendimiento y carga con JMeter identificando tiempos de respuesta y puntos críticos del sistema
4. Analizar la calidad de código con SonarQube reportando métricas de seguridad, mantenibilidad y cobertura.
5. Documentar un plan de pruebas integral según la norma ISO/IEC 29119
6. Implementar un pipeline de CI/CD con Docker y GitHub Actions para automatizar la ejecución de todas las pruebas.





DESARROLLO

3.1 Marco Teórico

3.1.1 Calidad de software ISO/IEC 25010

La norma ISO/IEC 25010 define un modelo de calidad para productos de software compuesto por 8 características principales: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad. Cada característica se descompone en sus características medibles que permiten resolver de manera objetiva la calidad de un sistema.

En el GymSupp Store se destaca en la adecuación funcional (el sistema realiza las operaciones CRUD), la fiabilidad (las pruebas unitarias verifican el comportamiento correcto), la seguridad (análisis de vulnerabilidades con SonarQube) y la mantenibilidad (métricas de code smells).

3.1.2 Procesos de Prueba ISO/IEC 29119

La norma ISO/IEC 29119 maneja procesos, documentación y técnicas para las pruebas de software, estableciendo que un proceso de prueba completo debe incluir planificación, diseño, implementación, ejecución y reporte de resultados.

Este proyecto se desarrollo con el plan de pruebas definiendo el alcance y estrategia, la especificación documenta cada caso con precondiciones y postcondiciones, y el reporte de ejecución registra los resultados obtenidos.

3.1.3 QA VS QC

Para el QA se enfoca en el proceso de desarrollo y prevención de defectos convirtiéndose en un proactivo con el fin de asegurar que los estándares de calidad se cumplan definiendo los procedimientos, formación y auditorias. Mientras que el QC se enfoca en la ejecución de pruebas y detección de defectos en el código, siendo un reactivo debido a que asegura que el producto final cumpla los requisitos incluyendo inspección, pruebas de software y revisión.



El aseguramiento de la calidad, conocido por sus siglas en inglés QA (Quality Assurance), es un conjunto de actividades planificadas y sistemáticas que se implementan dentro de un sistema de calidad para asegurar el cumplimiento de los requisitos de calidad para un producto o servicio a lo largo de todo su desarrollo, de principio a fin. Su objetivo principal es prevenir defectos en los productos o servicios durante el proceso de desarrollo, más que identificarlos una vez que hayan sido producidos.

El QA tester es indispensable para identificar y solucionar problemas potenciales antes de que se conviertan en defectos reales, lo que significa un ahorro sustancial en costes y una inexorable mejora de la productividad.

El control de calidad, conocido por sus siglas en inglés QC (Quality Control), es un proceso mediante el cual se verifica que los productos o servicios finales cumplan con los requisitos y estándares de calidad establecidos. Se centra en identificar y corregir los defectos en los productos terminados o en las etapas finales del proceso de producción.

La importancia del QC radica en su papel como una última línea de defensa contra productos defectuosos. Aunque el QA puede minimizar la ocurrencia de errores, es inevitable que algunos problemas pasen desapercibidos. Aquí es donde el QC resulta esencial: identifica estos defectos antes de que los productos lleguen a los inversores y a los clientes finales.

En este proyecto, QA se manifiesta en la definición del plan de pruebas, la selección de herramientas y la configuración del pipeline CI/CD. QC se manifiesta en la ejecución de las pruebas unitarias, funcionales y de rendimiento.



3.1.4 Métricas de Calidad

En el proyecto las métricas de calidad se basaron en:

Cobertura de código: porcentaje de líneas ejecutadas durante las pruebas. Se midió con JaCoCo obteniendo 96% global.

Defect density: número de defectos por unidad de código. SonarQube identificó 9 issues en 333 líneas de código.

MTTR (Mean Time To Recovery): tiempo promedio para recuperar el sistema ante un fallo. Estimado en menos de 5 minutos gracias al script de Docker Compose que levanta todos los servicios automáticamente.

3.2 Descripción de la Aplicación GymSupp Store

Es una aplicación web desarrollado con el framework Spring Boot 2.x en Java 21. Permitiendo gestionar el inventario de una tienda de suplementos deportivos mediante operaciones CRUD sobre la entidad Product.

Capa	Tecnología	Versión
Backend	Spring Boot	2.x
Lenguaje	Java	21
Base de datos	H2 (en memoria)	2.x
Frontend	Thymeleaf	3.x
Seguridad	Spring Security	6.x
Construcción	Maven	3.x

Tabla 1 Tecnologías usados en GymSupp Store

La arquitectura sigue el patrón MVC (Modelo-Vista-Controlador):

Model: clase Product con atributos id, name, description, price, stock, category -

Repository: ProductRepository extendiendo JpaRepository

Service: ProductService con lógica de negocio (save, findAll, findById, delete) -

Controller: ProductController manejando las rutas HTTP

Views: plantillas Thymeleaf en src/main/resources/templates

La seguridad está implementada con Spring Security configurando dos usuarios: admin (rol Admin) y vendor (rol Vendedor), con autenticación basada en formulario y protección CSRF.



3.3.1 Estrategia y Diseño

Las pruebas unitarias se implementaron usando Junit 5 con Mockito para el aislamiento de dependencias. Se desarrollaron dos clases de prueba: ProductServiceTest y ProductControllerTest

En ProductServiceTest verifica la lógica de negocio del servicio de maneta independiente, utilizando @ExtendWith y mockeando el repositorio con @Mock, esto es beneficioso porque las pruebas no dependen de la base de datos y se ejecutan en milisegundos.

Para ProductController utiliza @WebMvcTest con MockMvc para simular peticiones HTTP reales al controller sin levantar el servidor completo para que JaCoCo registre correctamente la cobertura del controller.

3.3.2 Resultados de Cobertura

GymSuppStore

GymSuppStore

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.gymsupp.store		37%		n/a	1	2	2	3	1	2	0	1
com.gymsupp.store.controller		95%		n/a	1	7	1	14	1	7	0	1
com.gymsupp.store.config		100%		n/a	0	10	0	31	0	10	0	1
com.gymsupp.store.service		100%		n/a	0	5	0	8	0	5	0	1
Total	7 of 228	96%	0 of 0	n/a	2	24	3	56	2	24	0	4

Ilustración 1. Reporte de cobertura JaCoCo

El paquete main contiene únicamente la clase GymSuppStoreApplication con el método main(), que no se usa en las pruebas unitarias



3.4 Pruebas Funcionales con Selenium WebDriver

3.4.1 POM (Patron Page Object Model)

El POM organiza las pruebas de interfaz separando la lógica de interacción con el navegador de la lógica de verificación, cada pagina de la aplicación se representa como una clase Java con métodos que encapsulan las acciones del usuario. Se implementaron dos Page Objects:

- LoginPage => Para encapsular el formulario de autenticación
- InventoryPage => Para encapsular la tabla de productos y sus acciones

Esto hace las pruebas mas seguras, si cambia el selector de un elemento HTML, solo se actualiza en un lugar.

Selenium es un proyecto general para una variedad de herramientas y bibliotecas que habilitan y soportan la automatización de navegadores web.

Proporciona extensiones para emular la interacción del usuario con los navegadores, un servidor de distribución para escalar la asignación del navegador, y la infraestructura para las implementaciones de la Especificación del controlador web W3C que le permite escribir código intercambiable para todos los principales navegadores web.



3.4.2 Casos de Prueba Ejecutados

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.04 s -- in Pruebas funcionales Selenium - GymSupp Store
[INFO] Results:
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco:0.8.11:report (report) @ store ---
[INFO] Loading execution data file C:\Users\USUARIO\Downloads\GymSuppStore\target\jacoco.exec
[INFO] Analyzed bundle 'GymSuppStore' with 4 classes
[INFO] BUILD SUCCESS
[INFO] Total time: 16.102 s
[INFO] Finished at: 2026-05-17T18:00:22-05:00
[INFO]
```

Ilustración 2. Ejecución correcta de pruebas Selenium en Chrome y Firefox

Los 5 casos cubrieron los flujos críticos: autenticación exitosa, autenticación fallida, visualización del inventario, creación de producto y verificación de listado. Todos ejecutaron exitosamente en Chrome y Firefox headless dentro del pipeline de GitHub Actions.

3.5 Pruebas de Rendimiento con JMeter

3.5.1 Plan de Prueba

En este punto se realizó dos escenarios principales:

Normal: 5 usuarios concurrentes con ramp-up de 5 segundos.

Carga: 50 usuarios concurrentes con ramp-up de 10 segundos.

Los endpoints probados fueron: GET y POST para login y edit.



3.5.2 Resultados

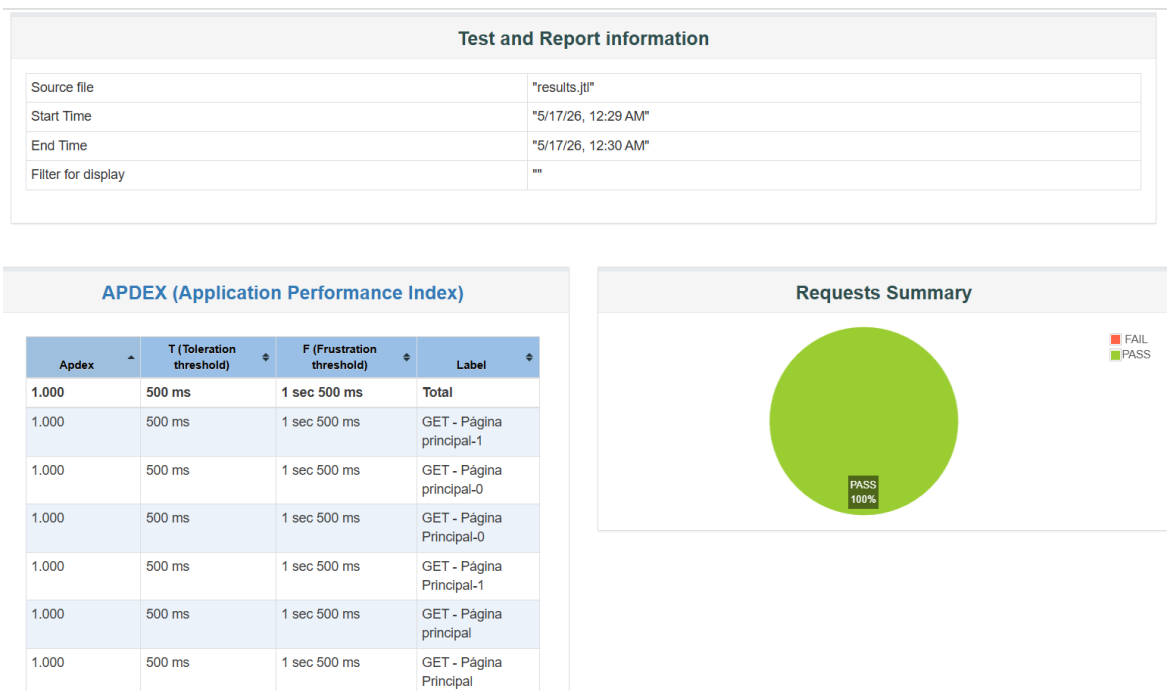


Ilustración 3. Dashboard de resultados JMeter

Los resultados muestran que la aplicación mantiene tiempos de respuesta aceptables bajo carga normal. Bajo carga alta se visualiza la degradación esperada por el uso H2 en memoria, lo cual se resolvería con PostgreSQL en producción.

3.6 Análisis de Calidad con SonarQube

3.6.1 Configuración

SonarQube Community Edition se ejecutó mediante Docker Compose, el análisis se realizó con el plugin sonar-maven-plugin integrando los resultados de Jacoco para la métrica de cobertura



3.6.2 Resultados del Dashboard

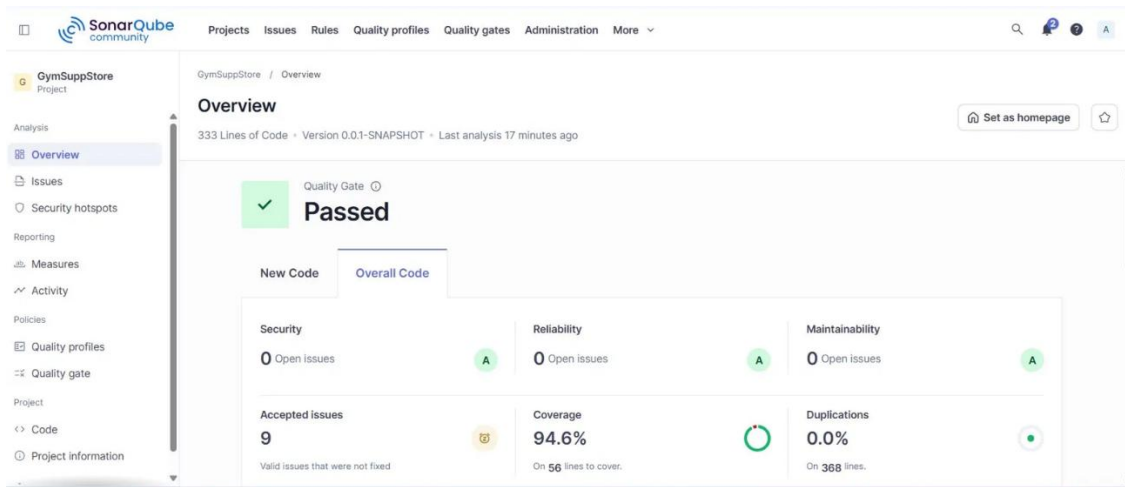


Ilustración 4. Dashboard SonarQube Quality Gate Passed

3.6.3 Hallazgos y Plan de mejora

SonarQube identificó 9 issues totales, todos aceptados y documentados:

Hallazgo 1 (Blocker): Credenciales hardcodeadas en SecurityConfig.java líneas 25 y 31. Las contraseñas gym123 y vend123 están escritas directamente en el código fuente. Plan de mejora: implementar UserDetailsService con usuarios almacenados en base de datos, contraseñas hasheadas con BCrypt y cargadas desde variables de entorno.

Hallazgo 2 (High): Entidad Product utilizada directamente en el controlador. Plan de mejora: implementar el patrón DTO (Data Transfer Object) para desacoplar la capa de presentación del modelo de dominio.

Hallazgo 3 (Medium): El literal String "redirect:/" aparece duplicado 3 veces en ProductController. Plan de mejora: definir una constante de clase private static final String REDIRECT_HOME = "redirect:/".



3.7 Docker y CI/CD con GitHub Actions

3.7.1 Docker Compose

La aplicación se orquesta mediante Docker Compose con los siguientes servicios: la aplicación Spring Boot (puerto 8080), la base de datos H2 embebida y SonarQube Community Edition (puerto 9000).

El Dockerfile utiliza construcción multi-stage: una etapa builder con eclipse-temurin:21-jdk-alpine para compilar y empaquetar, y una etapa final con eclipse-temurin:21-jre-alpine más liviana para ejecutar. Esto reduce el tamaño de la imagen final significativamente.

3.7.2 Pipeline GitHub Actions

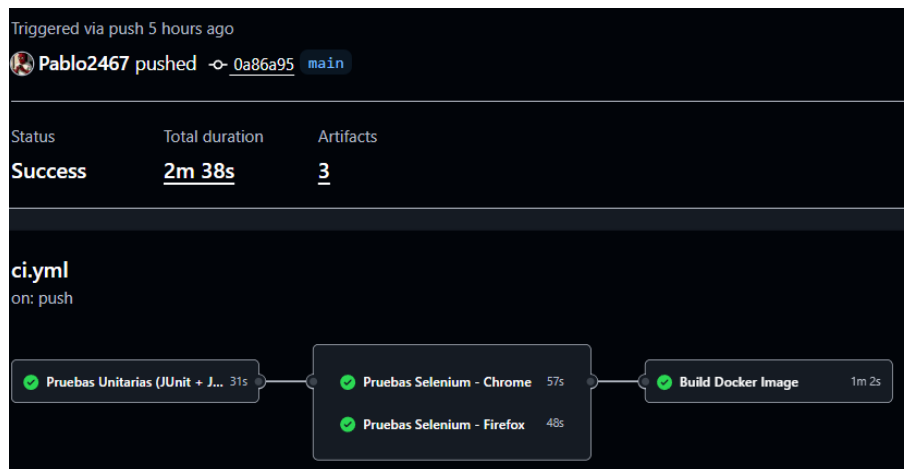


Ilustración 5. Pipeline CI/CD con todo ejecutado

Pruebas Unitarias: compila, ejecuta JUnit y genera reporte JaCoCo

Selenium Chrome: levanta la app y ejecuta los 5 casos en Chrome headless

Selenium Firefox: mismo proceso en Firefox headless (paralelo al anterior)

Build Docker: empaqueta y construye la imagen Docker final



En Chrome y Firefox se ejecutan de manera igual después de las pruebas unitarias, y el Docker espera a que ambos terminen de ejecutarse correctamente, donde el tiempo total de ejecución es de aproximadamente 3 minutos.

GitHub Actions es una plataforma de integración y despliegue continuos (IC/DC) que te permite automatizar tu mapa de compilación, pruebas y despliegue. Puede crear flujos de trabajo que compilen y prueben cada solicitud de incorporación de cambios en el repositorio o implementar solicitudes de incorporación de cambios combinadas en producción.

GitHub Actions va más allá de Solo DevOps y le permite ejecutar flujos de trabajo cuando se producen otros eventos en el repositorio. Por ejemplo, puedes ejecutar un flujo de trabajo para que agregue automáticamente las etiquetas adecuadas cada vez que alguien cree una nueva incidencia en tu repositorio.

GitHub proporciona máquinas virtuales Linux, Windows y macOS para ejecutar los flujos de trabajo, o puede hospedar sus propios ejecutores autohospedados en su propio centro de datos o infraestructura en la nube.

Puede configurar un GitHub Actions flujo de trabajo para que se desencadene cuando se produzca un evento en el repositorio, como cuando se abre una solicitud de incorporación de cambios o se crea una incidencia. El flujo de trabajo contiene uno o varios trabajos que se pueden ejecutar en orden secuencial o en paralelo. Cada trabajo se ejecutará dentro de su propio ejecutor de máquina virtual o dentro de un contenedor, y tendrá uno o varios pasos que pueden ejecutar un script que defina, o bien una acción, que es una extensión reutilizable que puede simplificar el flujo de trabajo.





Resultados Obtenidos

Criterio	Meta	Resultado	Estado
Junit Cobertura	$\geq 80\%$	96%	Aprobado
Casos Selenium	≥ 5	5/5	Aprobado
Selenium Chrome	Pasar	Passed	Aprobado
Selenium Firefox	Pasar	Passed	Aprobado
Quality Gate SonarQube	Passed	Passed	Aprobado
Security Rating	Aprobado	Aprobado	Aprobado
Tasa de error JMeter	0%	0%	Aprobado
Pipeline CI/CD	Verde	Verde	Aprobado
Docker Build	Excelente	Excelente	Aprobado

Todos los criterios de aceptación establecidos en el plan de pruebas fueron cumplidos o superados. La cobertura del 96% supera en 16 puntos porcentuales el mínimo exigido del 80%. El pipeline de CI/CD automatiza completamente la ejecución de todas las pruebas en cada push al repositorio, garantizando integración continua real.

Los hallazgos de SonarQube no representan defectos funcionales sino formas de mejorar para un ambiente de producción, el plan de mejora documentado en la sección 3.6.3 establece las acciones concretas para abordarlos.



CONCLUSION

El desarrollo de este proyecto permitió comprender de manera práctica la diferencia entre escribir código que funciona y escribir código de calidad. Las herramientas de QA/QC no son un paso adicional al desarrollo sino una parte integral del mismo que detecta problemas tempranos y reduce el costo de corrección.

La implementación de pruebas unitarias con JUnit 5 y Mockito demostró que el aislamiento de dependencias mediante mocking es fundamental para obtener pruebas rápidas, deterministas y con alta cobertura. El cambio de `@InjectMocks` a `@WebMvcTest` con `MockMvc` fue la decisión técnica más importante del proyecto, elevando la cobertura del controlador de 0% a 95%.

Selenium WebDriver con el patrón Page Object Model ofrece pruebas funcionales mantenibles. La separación entre la lógica de interacción y la lógica de verificación facilita adaptarse a cambios en la interfaz sin reescribir todos los tests. JMeter reveló que la aplicación tiene un rendimiento aceptable bajo carga normal pero que el uso de H2 en memoria limita la escalabilidad. Esto evidencia la importancia de probar con condiciones similares a producción. SonarQube demostró ser una herramienta indispensable al identificar problemas que el desarrollador no detecta visualmente: credenciales expuestas, violaciones de principios de diseño y código duplicado. El Quality Gate actúa como una puerta de calidad que impide integrar código problemático.

Finalmente, la automatización mediante GitHub Actions garantiza que cada cambio en el código pase por todas las pruebas antes de considerarse integrado, implementando el principio de integración continua de manera real y funcional.





Referencias

- Apache Software Foundation. (2023). Apache JMeter user manual (5.6.3).
<https://jmeter.apache.org/usermanual/> International Organization for Standardization. (2011). ISO/IEC 25010: Systems and software engineering
Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. ISO. International Organization for Standardization. (2022).
ISO/IEC 29119: Software and systems engineering — Software testing. ISO. JaCoCo. (2023).
JaCoCo Java code coverage library (0.8.11). <https://www.jacoco.org/jacoco/> JUnit Team. (2023). JUnit 5 user guide. <https://junit.org/junit5/docs/current/user-guide/> Selenium Project. (2023).
Selenium WebDriver documentation (4.x). <https://www.selenium.dev/documentation/> SonarSource. (2024). SonarQube documentation — Community Edition. <https://docs.sonarqube.org/> Spring. (2023).
Spring Boot reference documentation (2.x). <https://docs.spring.io/spring-boot/docs/current/reference/html/> Docker Inc. (2024). Docker documentation. <https://docs.docker.com/>
GitHub. (2024). GitHub Actions documentation. <https://docs.github.com/actions>
<https://tecnologia.euroinnova.com/qa-qc-diferencias>

